



Sunbeam Institute of Information Technology
Pune and Karad

Data structures and Algorithms

Trainer - Devendra Dhande

Email – devendra.dhande@sunbeaminfo.com

- **Course Schedules**
 - 21st July 2026 to 14th Aug 2026
 - Mon-Fri: Lecture – 5:00 PM to 8:00 PM
- **Course Format**
 - Participants are encouraged to code alongside (copy code from code-sharing utility in student portal).
 - Post your queries in chat box (on logical end of each topic).
 - Practice assignments will be shared. They are optional. If any doubts, share on WA group (possibly with screenshot). Faculty members or peers can help.
- **Course Goals**
 - Implement each DS & Algorithms from scratch.
 - Understand complexity of algorithms.
- **Programming language**
 - DS & Algorithms are language independent.
 - Classroom coding will be in Java (use IDE of your choice).
 - Will share C++/Python codes at the end of session.
- **Resource sharing**
 - <https://github.com/sunbeam-modular/dsa-o-13.git>
 - Recorded videos will be available on <http://students.sunbeamapps.org>

- **Java**

- ✓• Language Funda
- ✓• Methods
- ✓• Class & Object
- ✓• static members
- ✓• Arrays
- ✓• Collections

- **Python**

- Language Funda
- Functions
- Class & Object
- Collections

- **C++**

- Language Funda
- Functions
- Class & Object
- Friend class
- Arrays
- Pointers

- **C**

- Language Funda
- Functions
- Structures
- Arrays
- Pointers

- **Data Structures:**

- ✓ • Linked list
- { • Stack
- { • Queue
- ✓ • Hash Table
- { • Binary search tree
- { • Heap
- { • Graph

- **Algorithms:**

- Sorting
- Searching
- Stack, Queue & Linked list applications
- Graph algorithms.

- **Algorithm Design Techniques:**

- Brute Force
- Divide and Conquer
- Backtracking
- Greedy
- Dynamic Programming

- **Problem solving patterns:**

- Two/three pointer technique
- Fast and Slow pointers
- Sliding window (Fixed / Variable)
- Binary search
- BFS and DFS based
- Kth element category

- organising data inside memory for efficient processing.
- operations like add, delete, search, etc which can be performed on data.
- eg stack - push/pop/peek

- data structures are used to achieve
 - Abstraction
 - Reusability
 - Efficiency
- Due abstraction, all data structures are called as Abstract Data Types (ADT)
- Efficiency of a program/algorithm is measured in two parameters:
 1. Time - required to execute
 2. Space - taken inside memory to process input

Physical data structures



- tells about physical arrangement of data inside memory.
- e.g. Array, Linked List

(Basic)

Linear data structures

- data is organised sequentially/ linearly



- data can be accessed sequentially
- e.g. Array stack
 struct/class queue
 linked list

Hash Table

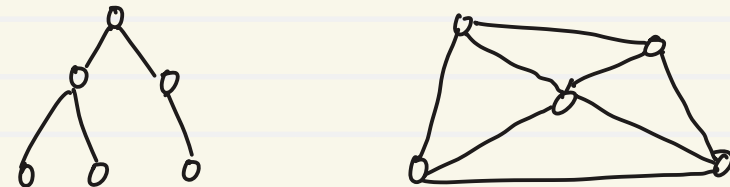
Logical data structures

- tells about relation betⁿ multiple data
- logical data structures are always implemented with physical structure.
- e.g. stack, queue, tree, heap, Graph

(Advanced)

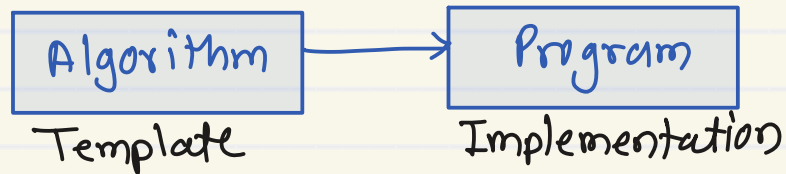
Non linear data structures

- data is organised in multiple levels (hierarchy)



- data can not be accessed sequentially
- e.g. Tree, Graph, Heap

- Algorithm is set of instructions given to the programmers
- Algorithm is step by step solution of given problem statement
- As written in human understandable languages, algorithms are programming languages independent
- Algorithms are used as templates and can be



Program : set of instructions to machine (CPU)
Algorithm: set of instructions to human (programmers)

Problem - Find sum of array elements

Algorithm:

1. Create space inside memory to store sum and initialize it to 0.
2. traverse array/collection from start to end (0 to N-1)
3. add each element into sum
4. return/print final sum

e.g. searching , sorting
linear binary selection bubble insertion...

- it is done for efficiency measurement and also known as time/space complexity
- It is done to finding time and space requirements of the algorithm
 1. Time - time required to execute the algorithm (ns, μ s, ms...)
 2. Space - space required to execute the algorithm inside memory (Bytes, kB, MB...)
- finding exact time and space of the algorithm is not possible because it depends on few external factors like
 - time is dependent on type of machine (CPU), number of processes running at that time
 - space is dependent on type of machine (architecture), data types

- finding growth rate by which time & space will get change when input will be change.

- Approximate time and space analysis of the algorithm is always done
- Mathematical approach is used to find time and space requirements of the algorithm and it is known as "Asymptotic analysis"
- Asymptotic analysis also tells about behaviour of the algorithm for different input or for change in sequence of input
- This behaviour of the algorithm is observed in three different cases
 1. Best case
 2. Average case
 3. Worst case

- to denote space & time complexity, we use asymptotic notations

① Omega (Ω)

↓
lower
bound

② Big O (O)

↓
Upper
bound

③ Theta (Θ)

↓
Avg / tight
bound

Time complexity

- time is directly proportional to number of iterations of the loops used in an algorithm
- To find time complexity/requirement of the algorithm count number of iterations of the loops

1. Find factorial of a number

```
fact = 1
for(i = 1; i <= n; i++)
    fact *= i
sysout(fact)
```

No. of iterations = n

Time $\propto n$

$$T(n) = O(n)$$

← linear growth

2. Print 2D array on console

```
for(i = 0; i < m; i++) {
    for(j = 0; j < n; j++) {
        sysout(arr[i][j]);
    }
}
```

No. of iterations of outer loop = m

No. of iterations of inner loop = n

Total iteration = $m * n$

Time $\propto m * n$

$$T(m, n) = O(m * n)$$

quadratic growth

$$m \approx n$$

$$\text{Time} \propto n * n$$

$$\text{Time} \propto n^2$$

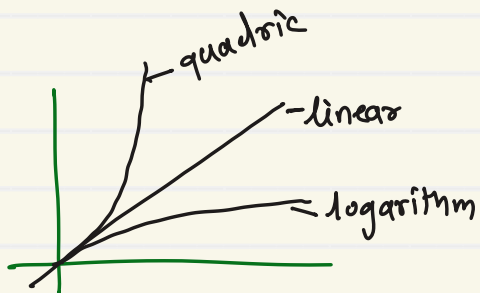
$$T(n) = O(n^2)$$

3. Print binary of decimal number

$$\begin{array}{r|l} 2 & 9 \\ \hline & 4 \\ \hline & 2 \\ \hline & 1 \end{array} \quad \begin{array}{l} 1 \\ 0 \\ 0 \\ 1 \end{array} \quad (9)_{10} = (1001)_2$$

```
while(n > 0) {
    rem = n % 2;
    n = n / 2;
}
```

n	n > 0	n % 2
9	T	1
4	T	0
2	T	0
1	T	1
0	F	



$$n = 9, 4, 2, 1$$

$$= n, n/2, n/4, n/8 \dots$$

$$= \frac{n}{2^0}, \frac{n}{2^1}, \frac{n}{2^2} \dots \dots \frac{n}{2^{\text{itr}}}$$

$\therefore$ for $n=1$ last time loop is executed

$$\frac{n}{2^{\text{itr}}} = 1$$

$$2^{\text{itr}} = n$$

$$\log 2^{\text{itr}} = \log n$$

$$\text{itr} \log 2 = \log n$$

$$\text{itr} = \frac{\log n}{\log 2}$$

$$\text{Time} \propto \frac{1}{\log 2} \log n$$

logarithm growth $\rightarrow$

$$T(n) = O(\log n)$$

4. Print table of given number

```
for (i=1 ; i<=10 ; i++)  
    sysout (n * i)
```

No. of iterations = 10 (fixed)

- loop iterations are fixed and not dependent on input
- constant time requirement, it is denoted as

$$T(n) = O(1) \leftarrow \text{constant time}$$

5. Find sum of two numbers

```
res = n1 + n2;  
sysout (res)
```

- irrespective of input values, will take same amount time always.
- constant time requirement, it is denoted as

$$T(n) = O(1)$$

Time complexity

Time complexities : $O(1)$, $O(\log n)$, $O(n)$, $O(n \log n)$, $O(n^2)$, $O(n^3)$, ..., $O(2^n)$,

Modification : + or - : time complexity is in terms of n

Modification : * or / : time complexity is in terms of $\log n$

$\text{for}(i=0; i < n; i++) \rightarrow O(n)$

$\text{for}(i=n; i > 0; i--) \rightarrow O(n)$

$\text{for}(i=0; i < n; i+=20) \rightarrow O(n)$

$\text{for}(i=n; i > 0; i/=2) \rightarrow O(\log n)$

$\text{for}(i=1; i < n; i*=2) \rightarrow O(\log n)$

$n = 8, 4, 2, 1$

$i = 1, 2, 4, 8$

$\text{for}(i=1; i \leq 10; i++) \rightarrow O(1)$

$\text{for}(i=0; i < n; i++) \rightarrow n$
 $\text{for}(j=0; j < n; j++) \rightarrow n \rightarrow O(n^2)$

$\text{for}(i=0; i < n; i++) i \rightarrow n \rightarrow 2n \rightarrow O(n)$
 $\text{for}(j=0; j < n; j++) j \rightarrow n$

$\text{for}(i=0; i < n; i++) \rightarrow n$
 $\text{for}(j=n; j > 0; j/=2) \rightarrow \log n \rightarrow O(n \log n)$

- Finding approximate space requirement of the algorithm to execute inside memory

Total space

=

Input space

+

Auxiliary space

↑
Actual space
required to store
input

↑
Extra space
required to process
input

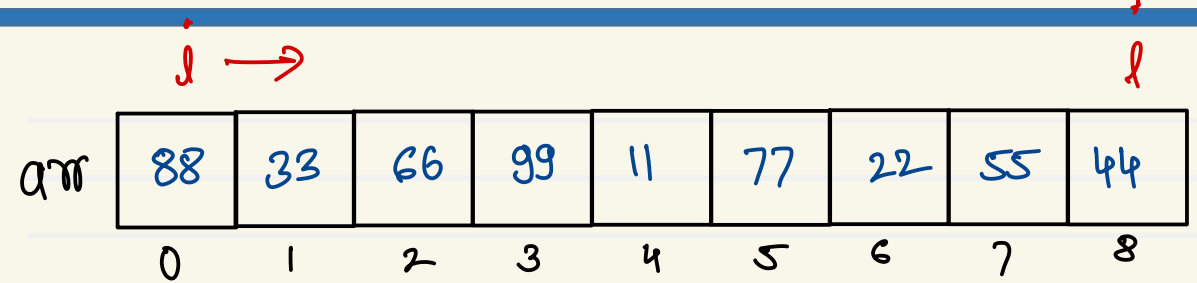
```
ArraySum(arr[]) {  
    sum = 1  
    for(i = 0 ; i < arr.length ; i++)  
        sum += arr[i];  
    return sum;  
}
```

input variable = arr
processing variables = sum, i
Auxiliary space = 2 units
↑
constant requirement

$$S(n) = O(1)$$

1. decide/take key from user
2. traverse collection of data from one end to another
3. compare key with data of collection
 - 3.1 if key is matching
return index/true
 4. if key is not matching
return -1/false

```
int linearSearch(arr, key) {  
    for( i=0; i < arr.length; i++) {  
        if( key == arr[i] )  
            return i;  
    }  
    return -1;  
}
```



$$\text{key} == \text{arr}[i]$$

77
Key

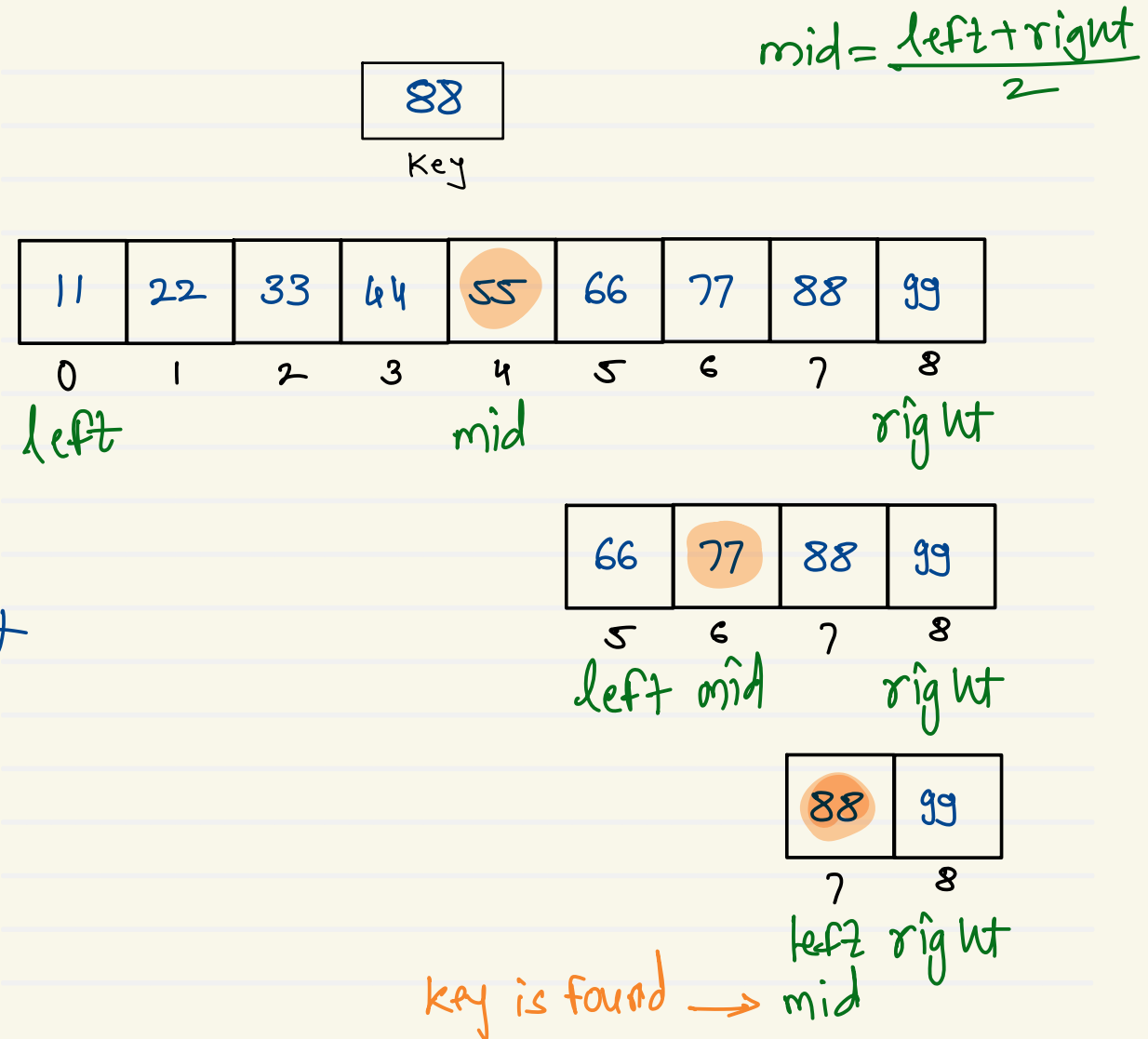
$i=5 \rightarrow$ key is found

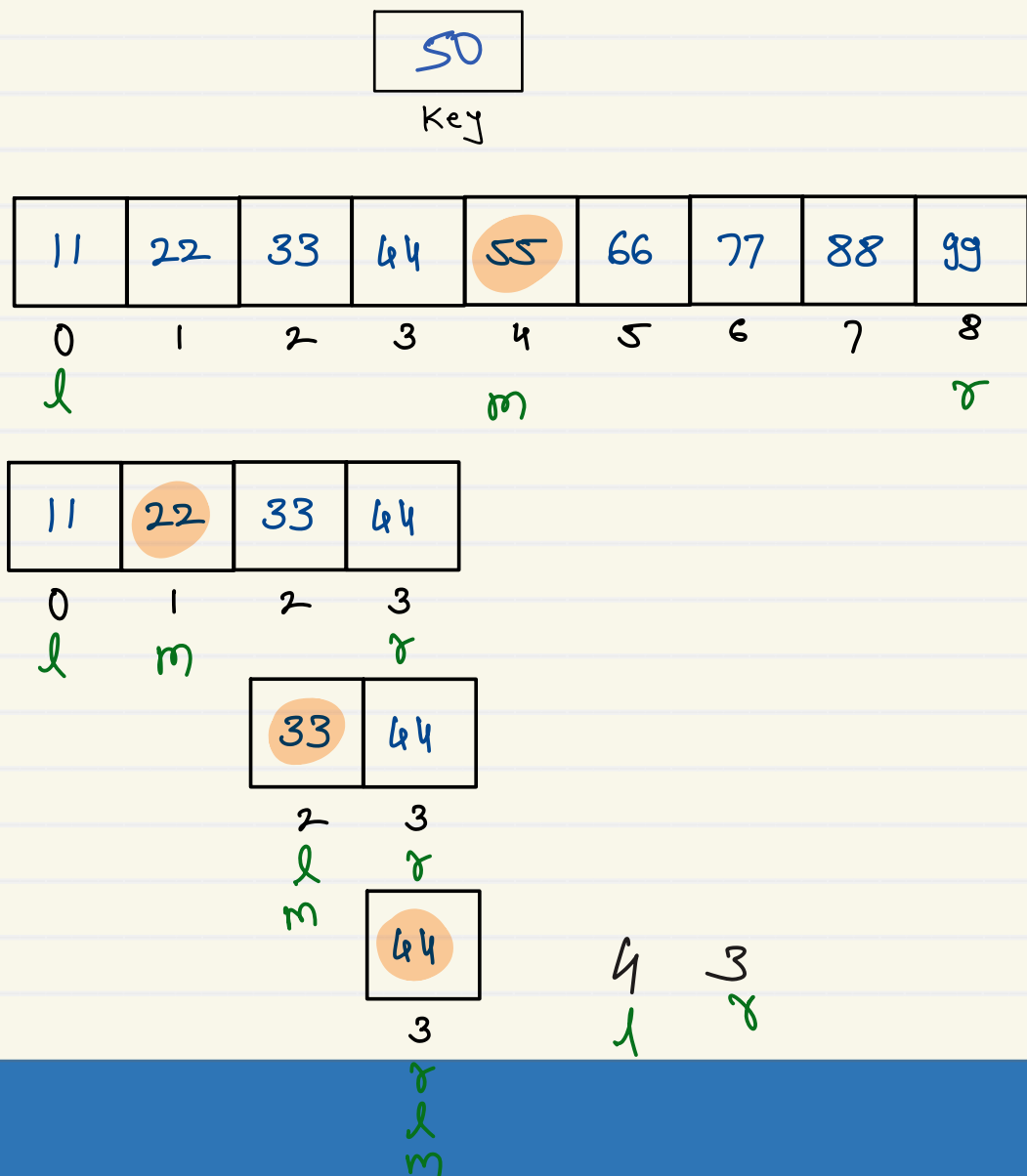
89
Key

$i=9 \rightarrow$ key is not found

Binary search

1. take key from user
2. divide array into two parts
(find middle element)
3. compare middle element with key
 - 3.1 if key is matching
return index (mid)
 - 3.2 if key is less than middle element
search key in left partition
 - 3.3 if key is greater than middle element
search key in right partition
- 4 if key is not matching
return -1





Array from left to right index

Left partition : left to mid-1

Right partition : mid+1 to right

valid partition : left $\leq$ right

invalid partition : left $>$ right

```

int binarySearch(arr, key) {
    l = 0, r = arr.length - 1, m;
    while (l <= r) {
        m = (l + r) / 2;
        if (key == arr[m])
            return m;
        else if (key < arr[m])
            r = m - 1;
        else
            l = m + 1;
    }
    return -1;
}

```

11	22	33	44	55	66	77	88	99
0	1	2	3	4	5	6	7	8

Key = 88

l	r	l <= r	m	
0	8	T	4	>
5	8	T	6	>
7	8	T	7	=

Key = 25

l	r	l <= r	m	
0	8	T	4	<
0	3	T	1	>
2	3	T	2	<
2	1	F		